

Procesamiento de Imágenes 1

Trabajo Práctico N.º 1

Informe académico

Materia	Procesamientos de Imágenes 1
Trabajo Práctico	Trabajo Práctico Nro 1
Docentes	Gonzalo Sad, Juan Manuel Calle, Joaquín Allione
Fecha	2 de Octubre de 2026
Integrantes	Alejandro Acosta, Diego Hauret, David Ibaceta, Gaston Rinaldi

El objetivo de este trabajo práctico es implementar y analizar dos técnicas de procesamiento digital de imágenes: la ecualización local de histograma y la extracción automatizada de información en actas de calificaciones. Para cada problema se describen el enfoque adoptado, las principales etapas del algoritmo, las dificultades encontradas y los resultados obtenidos.

Problema 1:

Desarrollo e Implementación del Algoritmo de Ecualización Local de Histograma

1. Introducción y Objetivo del Proyecto

Para resolver el problema del contraste deficiente en regiones oscuras o con información oculta en imágenes digitales (`Imagen_con_detalles_escondidos.tif`), diseñamos e implementamos desde cero un algoritmo de **Ecualización Local de Histograma (LHE)** mediante el uso de ventanas deslizantes (*sliding windows*).

A diferencia de la ecualización global, que redistribuye los niveles de gris basándose en el histograma acumulado de la imagen entera, el objetivo que nos planteamos fue **mejorar el contraste dinámico punto a punto**, revelando detalles y texturas ocultas en subregiones específicas sin saturar las áreas brillantes.

Hemos agregado un punto Bonus (en otro archivo) que es remplazar la fórmula para la ecualización del histograma por la función de `cv2.equalizeHist` ya que es mucho más rápida porque trabaja en C++.

2. Fundamentos Matemáticos del Algoritmo Desarrollado

Para la transformación de cada píxel central $f(i, j)$ dentro de una ventana vecindad de tamaño $M \times N$, aplicamos la función de distribución acumulada local (CDF).

La fórmula matemática que mapeamos en código para escalar el nivel de gris al rango $[0, 255]$ es:

$$T(v) = \text{round} \left(\frac{\text{CDF}(v) - \text{CDF}_{\min}}{(M \times N) - \text{CDF}_{\min}} \times 255 \right)$$

Donde:

- $v = f(i, j)$ es el nivel de gris del píxel central.
- $\text{CDF}(v)$ es la frecuencia acumulada del nivel de intensidad v dentro de la ventana $M \times N$.
- $\text{CDF}_{\min}$ es el valor mínimo no nulo de la función acumulada en dicha subregión.
- $(M \times N)$ representa la cantidad total de píxeles contenidos en la ventana.

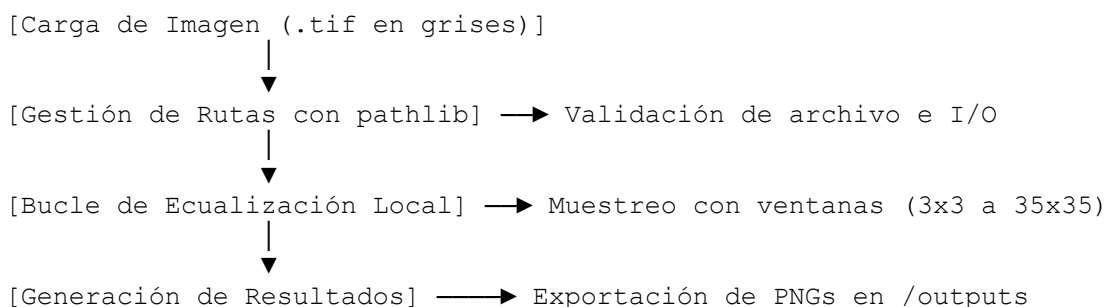
3. Desglose de la Implementación Algorítmica (`ecualizacion_local`)

Estructuramos la función `ecualizacion_local(img, M, N)` paso a paso para garantizar corrección matemática y manejo adecuado de bordes:

1. **Validación de Entradas:** Comprobamos que la matriz de entrada corresponda a una imagen bidimensional (escala de grises) y que las dimensiones del núcleo (M, N) sean valores estrictamente positivos.
2. **Tratamiento de Bordes (*Padding*):** Para evitar efectos de borde o la pérdida de dimensiones en la matriz resultante, utilizamos `cv2.copyMakeBorder()` con la estrategia `cv2.BORDER_REPLICATE`. Con esto extendemos los bordes replicando los valores límite a una distancia $\lfloor M/2 \rfloor$ y $\lfloor N/2 \rfloor$.
3. **Mapeo Ventana por Ventana:** Recorremos iterativamente la matriz de la imagen original y extraemos la submatriz correspondiente a la vecindad.
4. **Cálculo del Histograma y CDF Local:**
 - Empleamos `np.bincount()` fijando un vector de 256 niveles para computar el histograma en tiempo lineal.
 - Acumulamos las frecuencias con `np.cumsum()` para obtener la CDF local.
5. **Transformación adaptativa:** Obtenemos el valor del píxel central v . Evaluamos si la ventana carece de varianza (cuando $CDF_{\min} = M \times N$); de ser así, mantenemos el valor original. Caso contrario, aplicamos la ecuación de mapeo y aseguramos que el resultado permanezca en el intervalo válido $[0, 255]$ usando `np.clip()`.

4. Flujo de Ejecución y Pruebas Experimentales (`main`)

El script procesa la imagen de entrada evaluando diferentes escalas de ventana para analizar el compromiso entre detalle local y ruido:



1. **Carga y Verificación:** Leemos la imagen `Imagen_con_detalles_escondidos.tif` en modo escala de grises (`cv2.IMREAD_GRAYSCALE`) mediante rutas dinámicas relativas con `Path`.
2. **Evaluación Multi-Escala:** Iteramos la ejecución probando 8 configuraciones de ventanas impares (3×3 , 5×5 , 7×7 , 11×11 , 15×15 , 19×19 , 25×25 y 35×35).

3. **Exportación de Resultados:** Preparamos el almacenamiento ordenado en la carpeta `outputs/` exportando cada versión procesada a formato `.png`.

5. Análisis del Impacto del Tamaño de Ventana y Mejoras Futuras

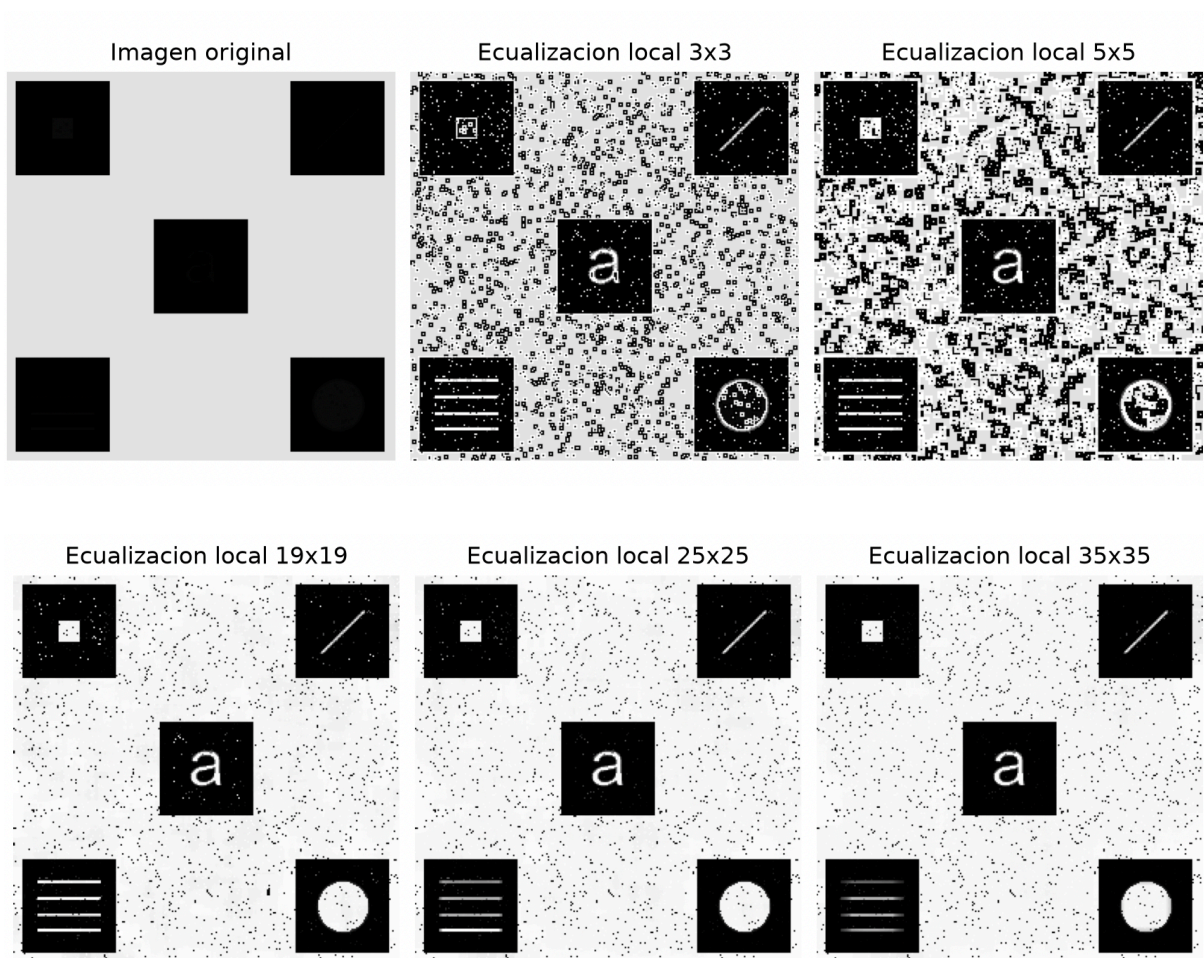
- **Efecto de Ventanas Pequeñas (3×3 a 7×7):** Realzan bordes y detalles muy finos, pero tienden a amplificar drásticamente el ruido de alta frecuencia en zonas homogéneas.
- **Efecto de Ventanas Grandes (19×19 a 35×35):** Proporcionan un balance óptimo, suavizando la transición de grises y aproximándose gradualmente a una ecualización global, aumentando el contraste sin generar artefactos visuales severos.

Lógicas de Optimización Vectorial (Futuras Versiones)

El algoritmo actual realiza un recorrido píxel por píxel con complejidad $\mathcal{O}(W \times H \times (M \times N))$. Para optimizar el tiempo de cómputo en imágenes de alta resolución, podríamos implementar:

- **Histogramas Integrales / Algoritmo de Ventana Deslizante**
Reutilizable: Actualizar el histograma local restando la columna que sale y sumando la que entra a medida que la ventana se desplaza, reduciendo la complejidad a $\mathcal{O}(W \times H)$.
- **Implementación de CLAHE (`cv2.createCLAHE`):** Alternativa que limita la amplificación de contraste (*Contrast Limited Adaptive Histogram Equalization*) y utiliza interpolación bilineal entre bloques para mayor eficiencia.

Resultados con distintas ecualizaciones:



Problema 2:

Desarrollo e Implementación del Sistema de Validación y Segmentación de Planillas de Calificaciones

1. Introducción y Objetivo del Proyecto

Para resolver el procesamiento automático de las planillas de calificaciones (`grade_sheet_X.png`), diseñamos e implementamos una solución basada en **Procesamiento Digital de Imágenes (PDI)** y reglas de **reconocimiento de patrones**.

El objetivo principal que nos planteamos fue automatizar la lectura de las hojas tabuladas, segmentar las celdas pertenecientes a cada estudiante (**Legajo, Nombre/Apellido, Parciales 1 a 3 y Condición Final**), validar la integridad de los datos ingresados según reglas geométricas y de conteo de componentes, y clasificar automáticamente las letras de condición final (**Aprobado, Recupera y Libre**).

2. Arquitectura de Datos y Organización del Código

Para mantener el código modular, mantenible y facilitar el manejo de las regiones de interés (ROIs), estructuramos la información en tres clases principales:

- **Record_crops:** La utilizamos para encapsular y gestionar los recortes (`crops`) en matriz de grises correspondientes a las seis celdas de un estudiante. Le incluimos un método `.save()` que nos permite exportar las imágenes a la carpeta `debug/` durante las pruebas.
- **Record_check:** Es la encargada de almacenar los estados de validación ("OK" o "MAL") de cada campo individual, evaluar la regla global del registro (todos los campos deben estar OK) y generar reportes formateados por consola.
- **Record:** Funciona como el objeto contenedor final que une la identidad del registro (`id`), sus imágenes (`Record_crops`) y su diagnóstico (`Record_check`).

3. Algoritmos de Procesamiento y Clasificación Desarrollados

A. Detección de Caracteres y Métricas Espaciales

Para aislar la información dentro de cada celda, desarrollamos la función `buscar_letras()`. Convertimos el recorte a escala de grises, lo binarizamos con inversión (`cv2.THRESH_BINARY_INV`) para dejar el trazo en blanco sobre fondo negro, y aplicamos componentes conectadas (`cv2.connectedComponentsWithStats`).

Con esto podemos filtrar ruido eliminando áreas menores a un umbral definido y extraer los *bounding boxes* (x, y, w, h) de cada carácter.

Posteriormente, con `contar_palabras()`, ordenamos vectorialmente los caracteres de izquierda a derecha a lo largo del eje X . Calculamos las distancias entre el final de una letra y el comienzo de la siguiente usando operaciones matriciales de NumPy; si la separación supera un umbral espacial prefijado, lo contabilizamos como un salto de palabra.

B. Clasificador Heurístico de Letras ("A", "R", "L")

En lugar de recurrir a una red neuronal pesada para clasificar la condición final, diseñamos un clasificador liviano y eficiente basado en la geometría del carácter (`clasificar_letra()`):

1. **Normalización:** Binarizamos y redimensionamos la letra extraída a una matriz estándar de **40x40 píxeles**.
2. **Detección de "L":** Analizamos el cuadrante superior derecho (`[0:20, 20:40]`). Si la densidad de píxeles activos es casi nula (menor a un umbral de ruido), concluimos que el carácter es una "L".
3. **Diferenciación "A" vs "R":** Para separar una "A" de una "R", medimos la simetría axial respecto al eje vertical. Reflejamos horizontalmente la mitad derecha (`cv2.flip`) y la restamos de la mitad izquierda con `cv2.absdiff`. Si la diferencia acumulada es muy baja (alta simetría), la clasificamos como "A"; si presenta asimetría por la estructura de la letra, la clasificamos como "R".

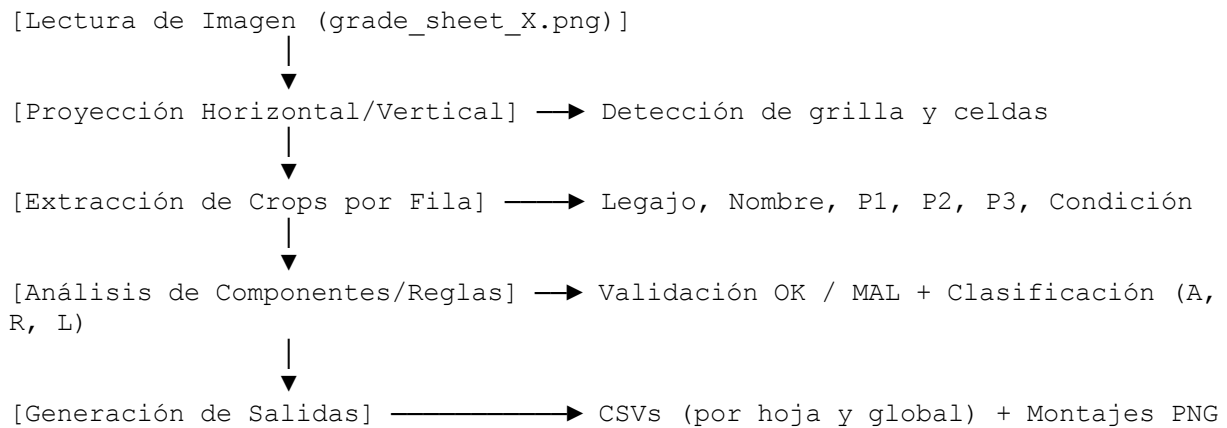
C. Reglas de Validación por Campo

Implementamos las comprobaciones específicas según los requerimientos de la planilla:

- **Legajo:** Comprobamos que la función de búsqueda detecte exactamente 8 componentes conectados.
- **Nombre y Apellido:** Exigimos que la celda contenga al menos 2 palabras (Nombre y Apellido) y no supere los 12 caracteres.
- **Parciales (1, 2 y 3):** Validamos que existan entre 1 y 2 caracteres numéricos consecutivos.
- **Condición Final:** Verificamos que exista un único carácter y le aplicamos el clasificador geométrico para determinar si el alumno es Aprobado, Recupera o Libre.

4. Flujo de Ejecución del Script (`main`)

El pipeline que programamos ejecuta los siguientes pasos secuenciales:



1. **Segmentación de la Grilla:** Cargamos las imágenes en escala de grises y aplicamos un umbralado para aislar las líneas negras de la tabla. Mediante proyecciones de suma de píxeles en los ejes horizontal ($axis=1$) y vertical ($axis=0$), detectamos los picos de intensidad que corresponden a los bordes de las celdas.
2. **Procesamiento y Recorte Fila por Fila:** Extraemos las subimágenes (crops) de cada celda aplicando márgenes de tolerancia de 2 píxeles para evitar incluir las líneas de la grilla en el análisis.
3. **Validación y Generación de Reportes:**
 - Exportamos los datos estructurados a archivos CSV (`validacion_grade_sheet_X.csv` y `validacion_resultados.csv`).
 - Construimos imágenes compuestas de salida utilizando NumPy y OpenCV para visualización rápida:
 - `*.valid.png` e `*.invalid.png`: Montajes organizados según el ancho y alto máximo de cada celda.
 - `no_aprobados.png`: Un reporte gráfico consolidado con tipografía `cv2.putText` que renderiza las observaciones de los estudiantes con condición "R" (Recupera) o "L" (Libre).

5. Posibles Líneas de Mejora para Futuras Versiones

- **Resiliencia ante Inclinación (*Deskew*):** La segmentación por proyecciones rígidas asume que la hoja está perfectamente alineada. Para mejorar la robustez ante escaneos rotados, se podría añadir una etapa previa de corrección de ángulo mediante la **Transformada de Hough** (`cv2.HoughLinesP`).
- **Sustitución de Heurísticas por ML/HOG:** Si bien la lógica de simetría y cuadrantes funciona bien con la tipografía probada, para soportar variaciones de manuscritos más complejas podríamos implementar la extracción de descriptores **HOG** (*Histogram of Oriented Gradients*) junto con un clasificador SVM o KNN.

Resultado no_aprobados.png (ejemplo):

Hoja 1 | Registro 1

BIANCA MONTE

Calificacion: L
Observacion: Libre

Hoja 1 | Registro 2

RIVAS HUGO

Calificacion: R
Observacion: Recupera

Hoja 1 | Registro 3

FARIAS JORGE

Calificacion: L
Observacion: Libre

Hoja 1 | Registro 4

JUANA GOMEZ

Calificacion: L
Observacion: Libre

Resultado validación_resultados.csv (ejemplo):

```
1 Hoja,Id,Legajo,Nombre_apellido,Parcial_1,Parcial_2,Parcial_3,Condicion_final
2 1,1,OK,OK,OK,OK,OK,OK
3 1,2,OK,OK,OK,OK,OK,OK
4 1,3,MAL,MAL,MAL,MAL,MAL,MAL
5 1,4,OK,OK,OK,OK,OK,OK
6 1,5,OK,OK,OK,OK,OK,OK
7 1,6,OK,OK,OK,OK,OK,OK
8 1,7,OK,OK,OK,OK,OK,OK
9 1,8,OK,OK,OK,OK,OK,OK
```